



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorio de docencia

Laboratorios de computación salas A y B

Profesor(a): Ing. Karina García Morales

Asignatura: Fundamentos de Programación (L)

Grupo: 22

No. de práctica(s): Práctica 9: Arreglos unidimensionales

Integrante(s): González Márquez José Luis

No. de lista o brigada: 20

Semestre: 2026-1

Fecha de entrega: Martes 4 de noviembre de 2025

Observaciones:

CALIFICACIÓN: _____

Objetivo: El alumnado elaborará programas en lenguaje FORTRAN para resolver problemas que requieran agrupar y almacenar una determinada cantidad de elementos del mismo tipo de dato numérico o carácter de forma consecutiva para su posterior procesamiento.

Desarrollo:

Arreglos (Arrays)

Un arreglo es una estructura de datos que agrupa **múltiples elementos del mismo tipo** (como int o char) en un solo bloque de memoria contiguo (uno al lado del otro).

Tamaño Fijo: Su tamaño (cuántos elementos puede guardar) se define al crearlo y no se puede cambiar después.

Acceso por Índice: Para leer o escribir en un elemento específico, se usa un **índice** (su número de posición).

Índices desde Cero: El primer elemento siempre está en el **índice 0**. Si un arreglo tiene n elementos, el último elemento está en el **índice n-1**

Los arreglos pueden ser **unidimensionales** (una sola fila, como lista[i]) o **multidimensionales** (como una tabla, tabla[fila][columna]).

Apuntadores (Pointers)

Un apuntador **no es una variable normal**. En lugar de guardar un dato (como 5 o 'a'), es una variable especial que **guarda la dirección de memoria de otra variable**.

Piensa en él como una nota que te dice *dónde* está guardado otro dato.

Para usarlos, hay dos operadores esenciales:

(Operador de Dirección):

Se usa para **obtener** la dirección de memoria de una variable.

ap = &c; significa: "Al apuntador ap asígnale la *dirección* de la variable c".

(Operador de Indirección o Dereferenciación):

Se usa para **acceder al dato** que está en la dirección a la que apunta el apuntador.

`printf("%c", *ap);` significa: "Imprime el *valor* que se encuentra en la dirección que *ap* está apuntando".

La Relación Clave: Arreglos y Apuntadores

Esta es la parte más importante:

En C, el nombre de un arreglo es un apuntador a su primer elemento.

Cuando declaras `int lista[5]`, la variable `lista` automáticamente funciona como un apuntador a la dirección de memoria de `lista[0]`.

Arreglos unidimensionales

Un arreglo unidimensional de n elementos se almacena en la memoria de la siguiente manera (Figura 1):



Figura 1. Almacenamiento en memoria de un arreglo unidimensional

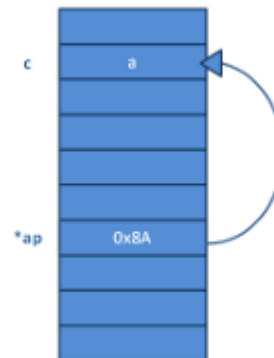


Figura 2. Un apuntador almacena la dirección de memoria de la variable a la que apunta

Conceptos:

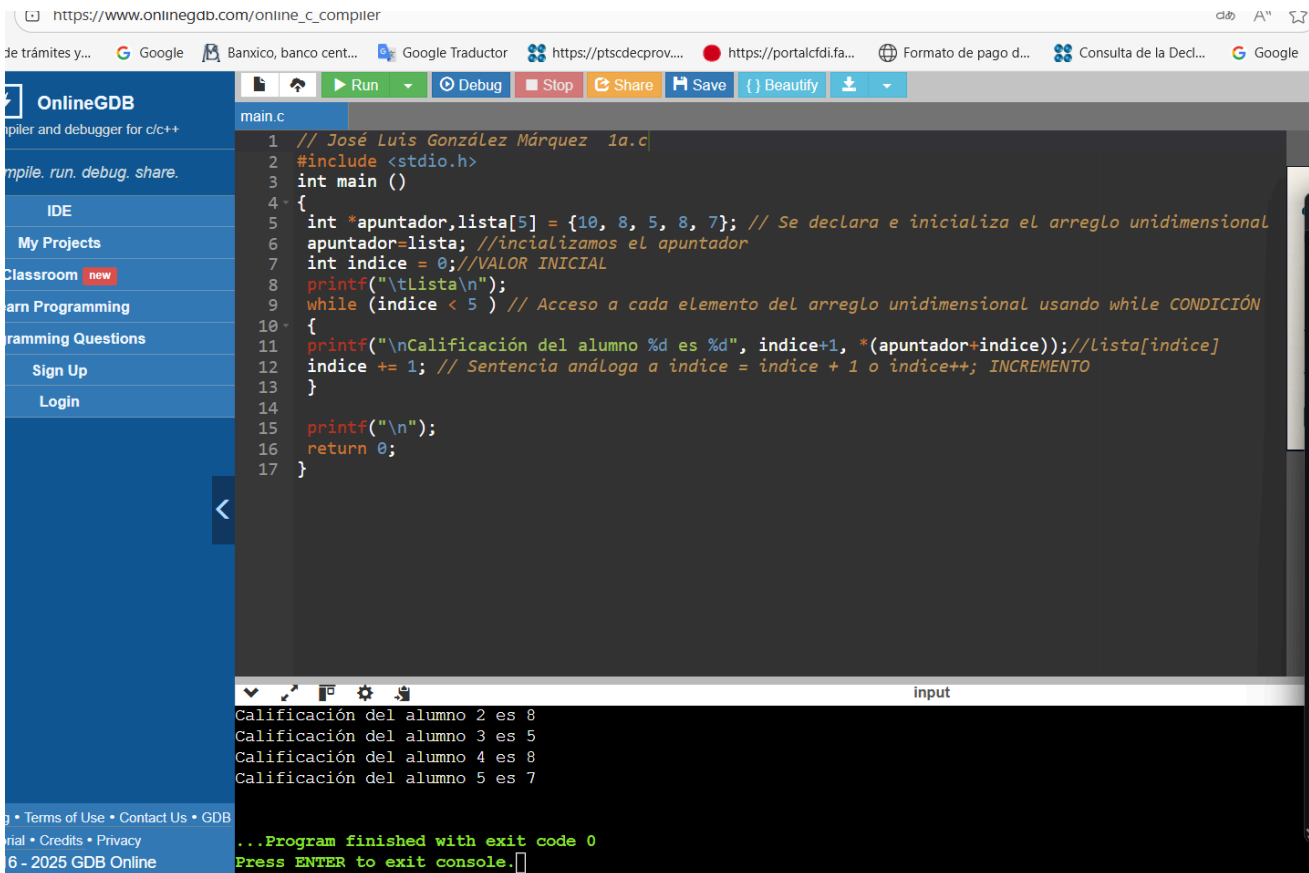
1. ¿Qué es un arreglo?

Un arreglo es un conjunto de datos contiguos del mismo tipo con un tamaño fijo definido al momento de crearse.

2. ¿Qué es un apuntador?

Un apuntador es una variable que contiene la dirección de una variable, es decir, hace referencia a la localidad de memoria de otra variable. Debido a que los apuntadores trabajan directamente con la memoria, a través de ellos se accede con rapidez a un dato.

3. Ejecutamos programa 1a.c y lo modificamos:



```
1 // José Luis González Márquez 1a.c
2 #include <stdio.h>
3 int main ()
4 {
5     int *apuntador, lista[5] = {10, 8, 5, 8, 7}; // Se declara e inicializa el arreglo unidimensional
6     apuntador=lista; //inicializamos el apuntador
7     int indice = 0; //VALOR INICIAL
8     printf("\tLista\n");
9     while (indice < 5 ) // Acceso a cada elemento del arreglo unidimensional usando while CONDICIÓN
10    {
11        printf("\nCalificación del alumno %d es %d", indice+1, *(apuntador+indice)); //Lista[indice]
12        indice += 1; // Sentencia análoga a indice = indice + 1 o indice++; INCREMENTO
13    }
14
15    printf("\n");
16    return 0;
17 }
```

input

Calificación del alumno 2 es 8
Calificación del alumno 3 es 5
Calificación del alumno 4 es 8
Calificación del alumno 5 es 7

...Program finished with exit code 0
Press ENTER to exit console.

Se revisa el cambio del primer programa con do-while y for

4. ¿Que hace el programa 2.c?

Se encarga de declarar un arreglo unidimensional de tamaño 10, pide al usuario cuántos elementos va a ingresar (entre 1 y 10), almacena números en el arreglo usando el ciclo for, y luego imprime todos los valores almacenados

5. Al programa2.c le vamos a aplicar apuntadores

```
1 // Jose Luis Gonzalez Marquez
2 #include <stdio.h>
3
4 int main ()
5 {
6     int lista[10]; // aqui arreglo unidimensional
7
8     int *ap = lista;
9
10    int indice = 0;
11    int numeroElementos = 0;
12
13    printf("\nDa un número entre 1 y 10 para indicar la cantidad de elementos que tiene el arreglo\n");
14    scanf("%d", &numeroElementos);
15
16    if((numeroElementos >= 1) && (numeroElementos <= 10))
17    {
18        // Usamos (ap + indice) es la direccion
19        printf("\n Captura de datos\n");
20        for (indice = 0 ; indice < numeroElementos ; indice++)
21        {
22            printf("Dar un número entero para el elemento %d del arreglo: ", indice );
23
24            // &lista[indice] pero Ahora: (ap + indice) porque (ap + indice) ya es la dirección de memoria
25            scanf("%d", (ap + indice));
26        }
27
28        printf("\nLos valores dados son: \n");
29
30        for (indice = 0 ; indice < numeroElementos ; indice++)
31        {
32            printf("%d ", *(ap + indice));
33        }
34    }
35    else
36    {
37        printf("el valor dado no es válido");
38    }
39
40    printf("\n");
41    return 0;
42 }
```

Da un número entre 1 y 10 para indicar la cantidad de elementos que tiene el arreglo
7

Captura de datos
Dar un número entero para el elemento 0 del arreglo: 2
Dar un número entero para el elemento 1 del arreglo: 2
Dar un número entero para el elemento 2 del arreglo: 4
Dar un número entero para el elemento 3 del arreglo: 5
Dar un número entero para el elemento 4 del arreglo: 7
Dar un número entero para el elemento 5 del arreglo: 8
Dar un número entero para el elemento 6 del arreglo: 9

Los valores dados son:
2 2 4 5 7 8 9

...Program finished with exit code 0
Press ENTER to exit console.

Aquí se agregaron apuntadores

6. Describe la sintaxis del arreglo

Un arreglo se declara así tipo nombre[tamaño];

7. Dar un ejemplo de arreglo unidimensional(explicito o implícito)

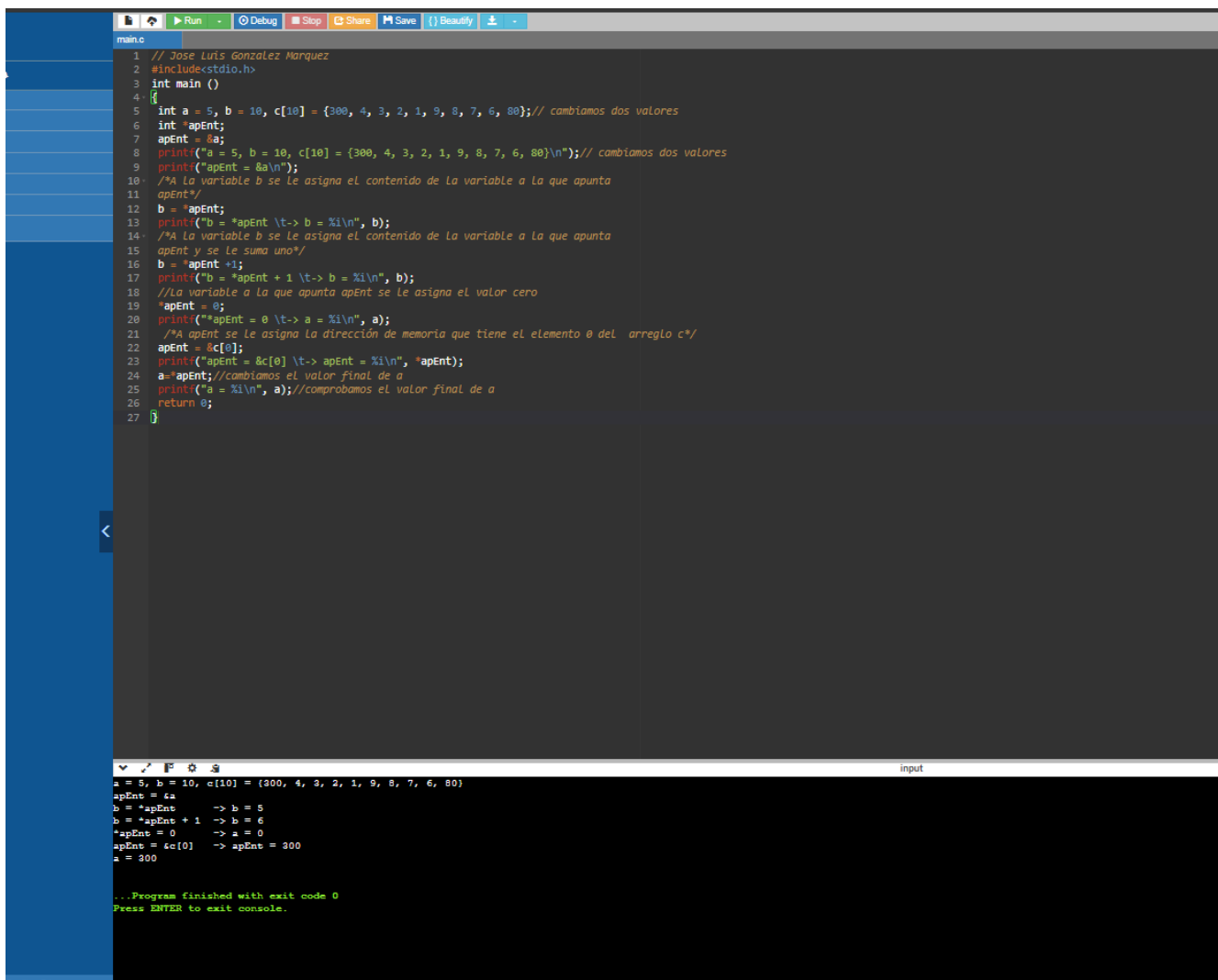
Explícito: int lista[9] = {3, 5, 9, 2, 1};

Uno Implícito (sin especificar tamaño, se deduce del inicializador): int arr[] = {75, 4, 3, 2, 1};

8. ¿Qué estructura requieres para manipular un arreglo por medio de sus índices?

Se requiere una estructura iterativa, for, while o do-while, para recorrer los índices del arreglo y acceder o modificar sus elementos.

9. Ejecución de programa 4.c con modificaciones



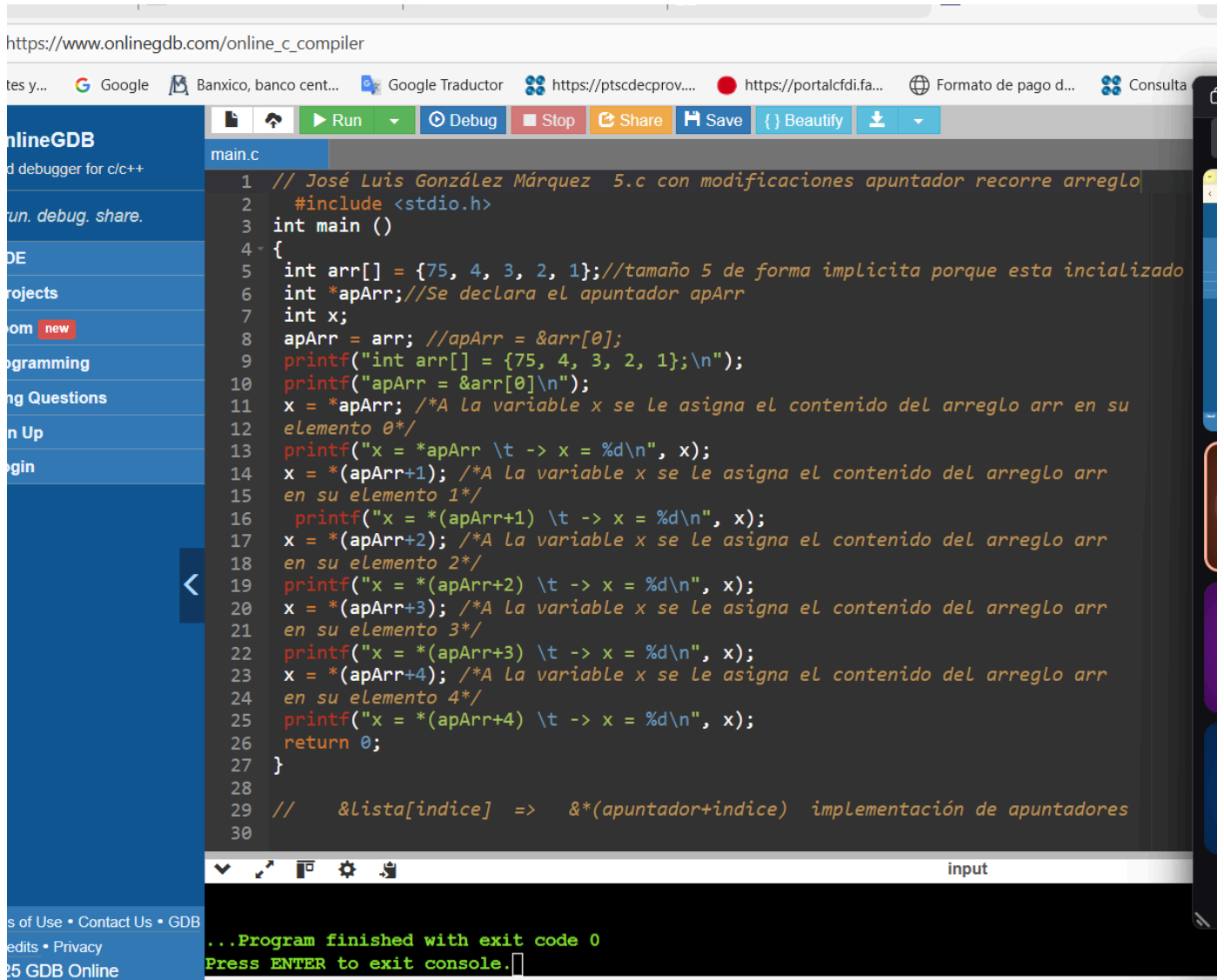
```
1 // Jose Luis Gonzalez Marquez
2 #include<stdio.h>
3 int main ()
4 {
5     int a = 5, b = 10, c[10] = {300, 4, 3, 2, 1, 9, 8, 7, 6, 80}; // cambiamos dos valores
6     int *apEnt;
7     apEnt = &a;
8     printf("a = 5, b = 10, c[10] = {300, 4, 3, 2, 1, 9, 8, 7, 6, 80}\n"); // cambiamos dos valores
9     printf("apEnt = %a\n");
10    /*A la variable b se le asigna el contenido de la variable a la que apunta
11    apEnt*/
12    b = *apEnt;
13    printf("b = *apEnt \t-> b = %i\n", b);
14    /*A la variable b se le asigna el contenido de la variable a la que apunta
15    apEnt y se le suma uno*/
16    b = *apEnt + 1;
17    printf("b = *apEnt + 1 \t-> b = %i\n", b);
18    /*La variable a la que apunta apEnt se le asigna el valor cero
19    *apEnt = 0;
20    printf("apEnt = 0 \t-> a = %i\n", a);
21    /*A apEnt se le asigna la dirección de memoria que tiene el elemento 0 del arreglo c*/
22    apEnt = &c[0];
23    printf("apEnt = &c[0] \t-> apEnt = %i\n", *apEnt);
24    a = *apEnt; //cambiamos el valor final de a
25    printf("a = %i\n", a); //comprobamos el valor final de a
26    return 0;
27 }
```

input

```
a = 5, b = 10, c[10] = {300, 4, 3, 2, 1, 9, 8, 7, 6, 80}
apEnt = 4a
b = *apEnt    -> b = 5
b = *apEnt + 1 -> b = 6
*apEnt = 0    -> a = 0
apEnt = &c[0] -> apEnt = 300
a = 300

...Program finished with exit code 0
Press ENTER to exit console.
```

10. Programa5.c que indica como trabaja un apuntador recorriendo un arreglo



```
https://www.onlinegdb.com/online_c_compiler

tes y... Google Banxico, banco cent... Google Traductor https://ptsdecprov... https://portalcfdi.fa... Formato de pago d... Consulta

onlineGDB
d debugger for c/c++
un. debug. share.
DE
projects
om new
programming
ng Questions
n Up
gin

main.c
1 // José Luis González Márquez 5.c con modificaciones apuntador recorre arreglo
2 #include <stdio.h>
3 int main ()
4 {
5     int arr[] = {75, 4, 3, 2, 1}; // tamaño 5 de forma implícita porque está inicializado
6     int *apArr; // Se declara el apuntador apArr
7     int x;
8     apArr = arr; // apArr = &arr[0];
9     printf("int arr[] = {75, 4, 3, 2, 1};\n");
10    printf("apArr = &arr[0]\n");
11    x = *apArr; /* A la variable x se le asigna el contenido del arreglo arr en su
12    elemento 0 */
13    printf("x = *apArr \t -> x = %d\n", x);
14    x = *(apArr+1); /* A la variable x se le asigna el contenido del arreglo arr
15    en su elemento 1 */
16    printf("x = *(apArr+1) \t -> x = %d\n", x);
17    x = *(apArr+2); /* A la variable x se le asigna el contenido del arreglo arr
18    en su elemento 2 */
19    printf("x = *(apArr+2) \t -> x = %d\n", x);
20    x = *(apArr+3); /* A la variable x se le asigna el contenido del arreglo arr
21    en su elemento 3 */
22    printf("x = *(apArr+3) \t -> x = %d\n", x);
23    x = *(apArr+4); /* A la variable x se le asigna el contenido del arreglo arr
24    en su elemento 4 */
25    printf("x = *(apArr+4) \t -> x = %d\n", x);
26    return 0;
27 }
28
29 // &lista[indice] => &*(apuntador+indice) implementación de apuntadores
30

input

...Program finished with exit code 0
Press ENTER to exit console.
```

11. Programa7.c (aplicamos el uso de Strlen

```
main.c
1 // Jose Luis Gonzalez Marquez
2 #include <stdio.h>
3 #include <string.h>
4 int main()
5 {
6     char palabra[20]; //arreglo tamaño 20
7     int i=0;
8     printf("Ingrese una palabra: ");
9     scanf("%s", palabra); /* Se omite & porque el propio nombre del arreglo de
10 tipo cadena apunta, es decir, es equivalente a la dirección de comienzo del
11 propio arreglo*/
12     printf("La palabra ingresada es: %s\n", palabra); // si fuera caracter sería %c
13     // este for me sirve para imprimir la palabra que ingresó el usuario
14     for (i = 0 ; i<strlen(palabra) ; i++) // i < 20      i<strlen(palabra-1)  (n-1)
15     {
16         printf("%c", palabra[i]); // %c porque va a recorrer letra por letra
17     }
18     return 0;
19 }
20
21 // &lista[indice] => &(apuntador+indice) implementación de apuntadores
```

Ingrese una palabra: Tacos
La palabra ingresada es: Tacos
Tacos

...Program finished with exit code 0
Press ENTER to exit console.

Tarea:

1.-Indica que realiza el siguiente programa

```
#include <stdio.h>

int arreglo[] = {16,32,42,67,25,20,73,28,17,45};

int i, j, n, aux;

int main() {

    n = 10;

    for(i=1; i< i++)

    {

        j = i;

        aux = arreglo[i];

        while(j>0 && aux<arreglo[j-1])

        {

            arreglo[j] = arreglo[j-1];

            j=j-1;

        }

        arreglo[j] = aux;

    }

    printf("\n\nLos elementos obtenidos del arreglo son: \n");

    for(i=0; i< i++)

    {

        printf("Elemento [%d]: %d\n", i, arreglo[i]);

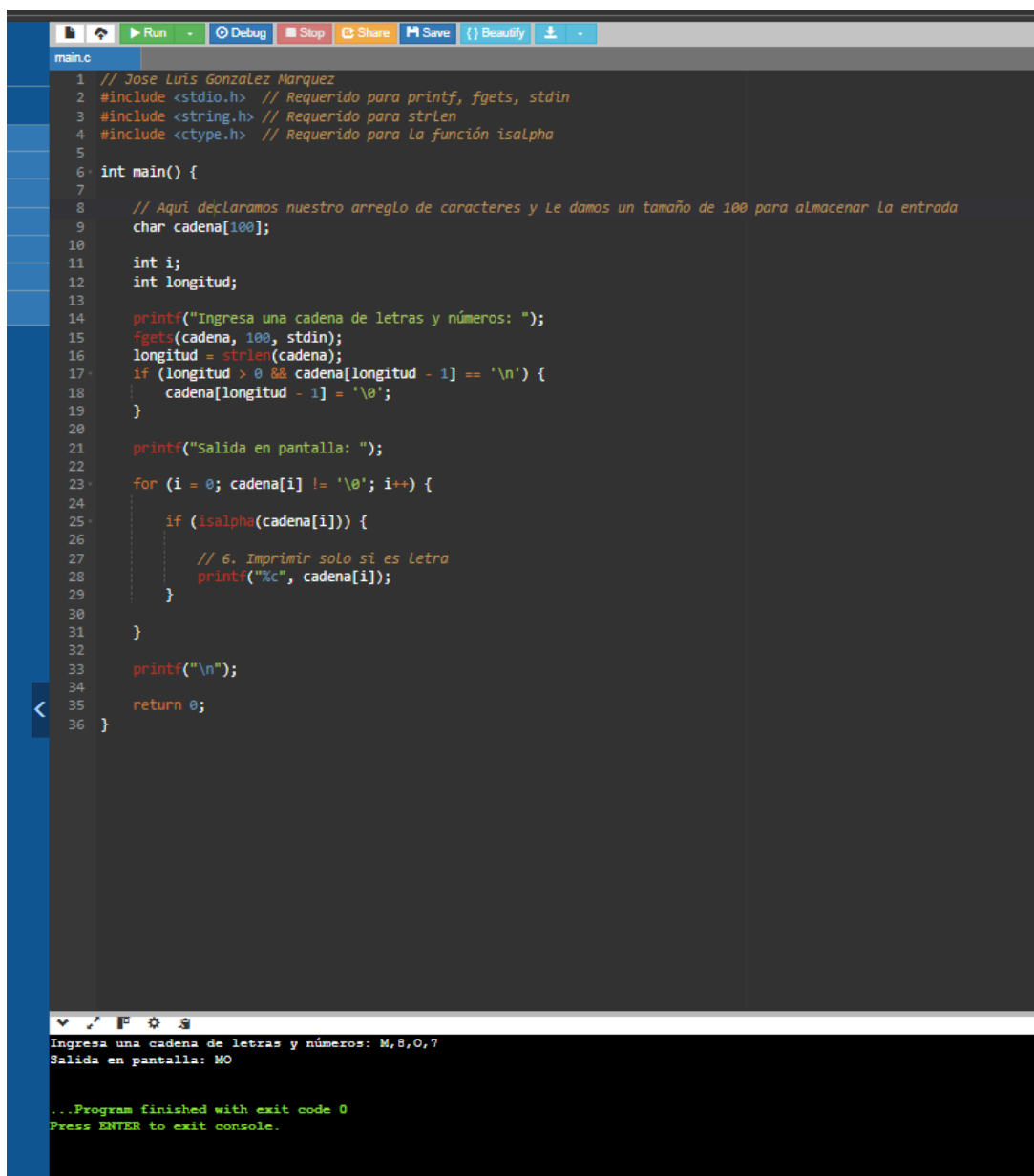
    }

    return 0;

}
```

El objetivo de este algoritmo es tomar el arreglo desordenado y ordenar de menor a mayor. Pero cuando intenté ejecutarlo me di cuenta que el for ($i = 1; i < i++$) hace que falle y no entra al ciclo. Para arreglarlo debe ser for($i = 1; i < n; i++$). En el segundo for está diseñado para recorrer el arreglo (que *debería* estar ya ordenado) e imprimir cada elemento en la pantalla, uno por uno, con su respectivo índice, pero también está mal: for($i = 0; i < i++$) Debe ser for ($i = 0; i < n; i++$).

2.- Genera un programa que le solicite una cadena de letras y números al usuario(emplear arreglo) e imprima solo letras. Este arreglo debe ser de caracteres, recuerda que un caracter puede almacenar números pero un arreglo de números no puede almacenar caracteres.



```
main.c
1 // Jose Luis Gonzalez Marquez
2 #include <stdio.h> // Requerido para printf, fgets, stdin
3 #include <string.h> // Requerido para strlen
4 #include <ctype.h> // Requerido para la función isalpha
5
6 int main() {
7
8     // Aquí declaramos nuestro arreglo de caracteres y le damos un tamaño de 100 para almacenar la entrada
9     char cadena[100];
10
11     int i;
12     int longitud;
13
14     printf("Ingresa una cadena de letras y números: ");
15     fgets(cadena, 100, stdin);
16     longitud = strlen(cadena);
17     if (longitud > 0 && cadena[longitud - 1] == '\n') {
18         cadena[longitud - 1] = '\0';
19     }
20
21     printf("Salida en pantalla: ");
22
23     for (i = 0; cadena[i] != '\0'; i++) {
24
25         if (isalpha(cadena[i])) {
26
27             // 6. Imprimir solo si es letra
28             printf("%c", cadena[i]);
29         }
30     }
31
32     printf("\n");
33
34     return 0;
35 }
36
```

Ingresa una cadena de letras y números: M,8,0,7
Salida en pantalla: MO

...Program finished with exit code 0
Press ENTER to exit console.

3.- Genera un programa que solicite al usuario un vector de 15 enteros haciendo uso de un arreglo y la estructura iterativa para recorrerlo por medio de sus índices e imprimir en pantalla.

```
main.c
1 // Jose Luis Gonzalez Marquez
2 #include <stdio.h>
3
4 int main() {
5
6     // Aquí va el vector
7     int vector[15];
8     int i;
9
10    printf("Ingresa 15 números entero\n");
11
12    // Aquí el for
13    for (i = 0; i < 15; i++) {
14        printf("Ingresa el valor para la posición [%d]: ", i);
15
16        // i para guardar el numero
17        scanf("%d", &vector[i]);
18    }
19
20
21    printf("\nMostrando el vector completo\n");
22
23    //iterativa
24    for (i = 0; i < 15; i++) {
25        printf("Elemento en la posición [%d] es: %d\n", i, vector[i]);
26    }
27
28    return 0;
29 }
```

```
Elemento en la posición [1] es: 2
Elemento en la posición [2] es: 3
Elemento en la posición [3] es: 4
Elemento en la posición [4] es: 5
Elemento en la posición [5] es: 6
Elemento en la posición [6] es: 7
Elemento en la posición [7] es: 8
Elemento en la posición [8] es: 9
Elemento en la posición [9] es: 10
Elemento en la posición [10] es: 11
Elemento en la posición [11] es: 12
Elemento en la posición [12] es: 13
Elemento en la posición [13] es: 14
Elemento en la posición [14] es: 15

...Program finished with exit code 0
Press ENTER to exit console.
```

4.- Modifica el programa (Programa7.c) y aplicar el uso de apuntadores.

```
#include <stdio.h>

#include <string.h>

int main()

{

    char palabra[20]; //arreglo tamaño 20

    int i=0;

    printf("Ingrese una palabra: ");

    scanf("%s", palabra); /* Se omite & porque el propio nombre del arreglo de
    tipo cadena apunta, es decir, es equivalente a la dirección de comienzo del
    propio arreglo*/

    printf("La palabra ingresada es: %s\n", palabra); // si fuera caracter sería %c

    // este for me sirve para imprimir la palabra que ingresó el usuario

    for (i = 0 ; i<strlen(palabra) ; i++) // i < 20    i<=strlen(palabra-1) (n-1)

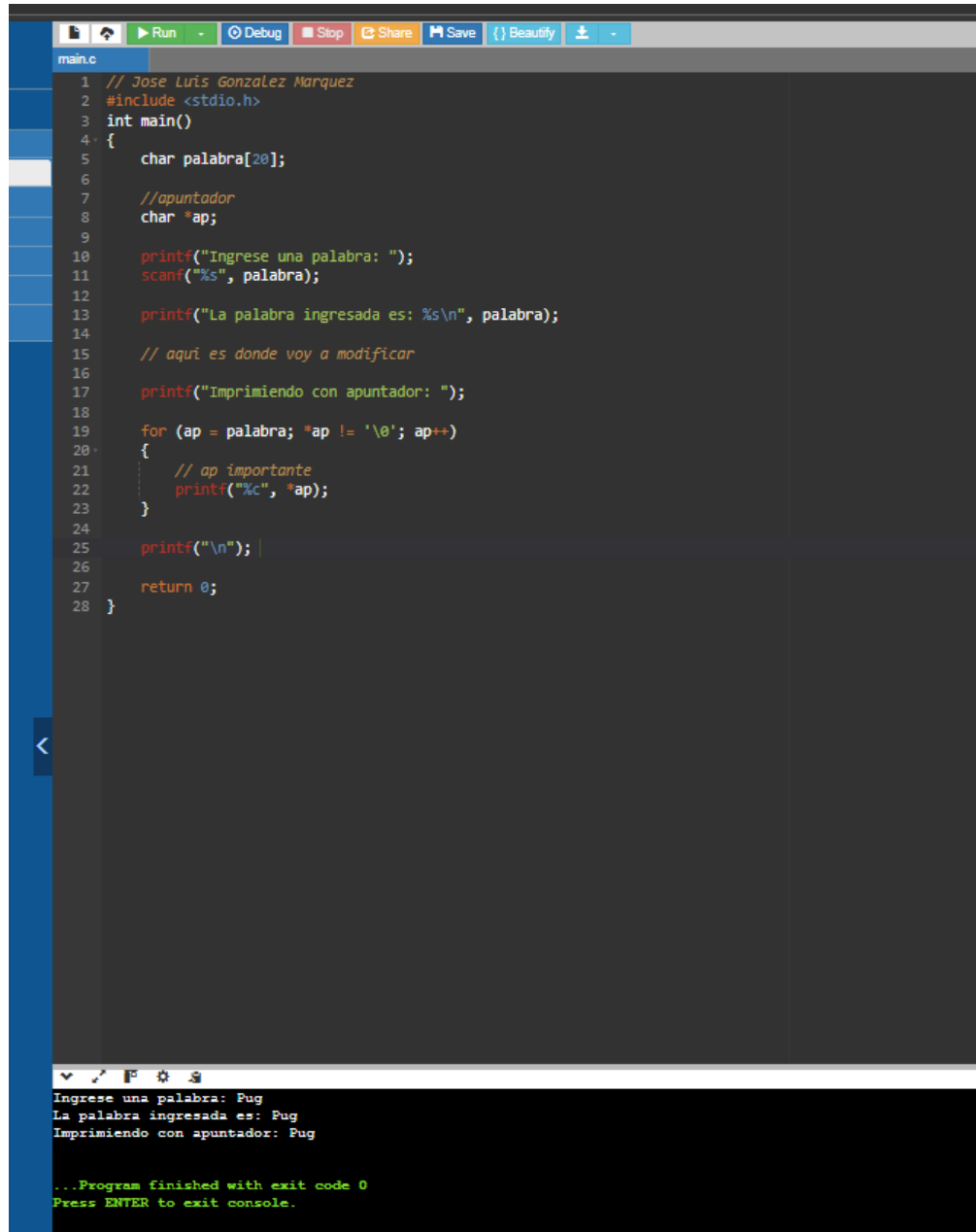
    {

        printf("%c", palabra[i]); // %c porque va a recorrer letra por letra

    }

    return 0;

}
```



The image shows a code editor window with a dark theme. The editor has a toolbar at the top with buttons for Run, Debug, Stop, Share, Save, and Beautify. The file name 'main.c' is visible in the top left. The code is a C program that prompts the user to enter a word, prints it, and then iterates through each character to print it individually. The terminal output at the bottom shows the program's execution with the input 'Pug'.

```
1 // Jose Luis Gonzalez Marquez
2 #include <stdio.h>
3 int main()
4 {
5     char palabra[20];
6
7     //apuntador
8     char *ap;
9
10    printf("Ingrese una palabra: ");
11    scanf("%s", palabra);
12
13    printf("La palabra ingresada es: %s\n", palabra);
14
15    // aqui es donde voy a modificar
16
17    printf("Imprimiendo con apuntador: ");
18
19    for (ap = palabra; *ap != '\0'; ap++)
20    {
21        // ap importante
22        printf("%c", *ap);
23    }
24
25    printf("\n");
26
27    return 0;
28 }
```

Ingrese una palabra: Pug
La palabra ingresada es: Pug
Imprimiendo con apuntador: Pug

...Program finished with exit code 0
Press ENTER to exit console.

Conclusión:

Mi conclusión va a empezar diciendo que me gusto mucho esta práctica, la razón es sencilla y es porque estamos introduciendo poco a poco las diferentes herramientas que vamos a utilizar y eso hace que podíamos agilizar pero al mismo tiempo aprender a aplicar cada una de las cosas que aprendemos porque la práctica siempre incluye ejemplos muy buenos y fáciles de ejecutar para todo. Además de eso la maestra nos ayuda siempre a ejecutar cada uno de los programas en clase para que podamos aprenderlo mejor, sinceramente eso ayuda bastante.

Fuentes:

Facultad de Ingeniería. (2025). Manual de prácticas del laboratorio de Fundamentos de Programación. Laboratorio de computación. Salas A y B. Universidad Nacional Autónoma de México. pp. 109-116. Recuperado el 2 de noviembre de 2025 de <http://lcp02.fi-b.unam.mx/>

[Tutorial de Fortran. \(s.f.\). Scribd. https://es.scribd.com/document/349942182/Tutorial-de-Fortran](https://es.scribd.com/document/349942182/Tutorial-de-Fortran)

[Universidad de Cantabria. \(s.f.\). 4 ARRAYS. OCW Universidad de Cantabria. https://ocw.unican.es/pluginfile.php/1825/course/section/1419/Curso-Fortran-4.pdf](https://ocw.unican.es/pluginfile.php/1825/course/section/1419/Curso-Fortran-4.pdf)